

Universidad de Concepción  
Facultad de Ingeniería  
Departamento de Ingeniería Informática y Ciencia de la Computación

## Proyecto Matemáticas Discretas Mapa turístico



José González Aguayo  
Mario Salgado Jaque  
Gonzalo Quilodrán Neira

12 de mayo de 2026

# Índice

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                    | <b>3</b>  |
| <b>2. Objetivos</b>                                       | <b>3</b>  |
| 2.1. Objetivos generales . . . . .                        | 3         |
| 2.2. Objetivos específicos . . . . .                      | 3         |
| <b>3. Modelo</b>                                          | <b>4</b>  |
| 3.1. Interpretación del archivo de texto (mapa) . . . . . | 4         |
| 3.2. ¿Cómo usaremos grafos? . . . . .                     | 4         |
| 3.3. Construir y almacenar el grafo . . . . .             | 4         |
| 3.4. Suposiciones sobre la idea de ruta . . . . .         | 5         |
| 3.5. Encontrar caminos dentro del grafo . . . . .         | 5         |
| 3.6. Dar instrucciones al turista . . . . .               | 5         |
| <b>4. Implementación</b>                                  | <b>7</b>  |
| 4.1. estructuras.h . . . . .                              | 7         |
| 4.2. lecturaMapas.c . . . . .                             | 7         |
| 4.3. construyeMapa.c . . . . .                            | 8         |
| 4.4. rutaTuristica.c . . . . .                            | 8         |
| 4.5. main.c . . . . .                                     | 9         |
| 4.6. Compilación . . . . .                                | 9         |
| <b>5. Conclusiones</b>                                    | <b>10</b> |



## 1. Introducción

Uno de los conocimientos principales en las matemáticas discretas, es la teoría de grafos y dígrafos, vértices conectados por aristas o arcos con una dirección fija, respectivamente. El uso de estos se pueden ver día a día, en redes sociales, en algoritmos, o mapas. Aquellas aplicaciones de mapas, que permiten buscar una ruta desde un punto A hasta un punto B, trabajan de fondo con dígrafos para poder trazar una ruta y que el usuario pueda llegar a su destino.

En eso consistirá este proyecto, el crear un código que al ingresarle un mapa, con distintas calles y puntos turísticos, y luego de analizarlo, que se le entregue al usuario una ruta de como llegar a cada uno de los puntos, dándole instrucciones precisas, dependiendo de la distancia entre puntos, el sentido de las calles y avisarle en caso de que haya algún problema al momento de calcular la ruta.

Los autores de este proyecto podrán pulir su conocimiento adquirido y lo podrán poner en práctica en el desarrollo de este proyecto, que será documentado en este archivo.

## 2. Objetivos

### 2.1. Objetivos generales

El objetivo del proyecto es desarrollar un código en lenguaje C que reciba un archivo txt con la descripción de un mapa de una ciudad, donde se denote calles en distintas direcciones (horizontales, verticales o diagonales), y además se incluya una lista de puntos turísticos. El código debe leer el archivo, interpretarlo y encontrar un camino entre los puntos para darle al usuario una ruta donde pueda recorrer todos los puntos de interés del mapa.

### 2.2. Objetivos específicos

- El programa puede leer el archivo que le entrega el usuario, y puede interpretarlo correctamente.
- El programa puede reconocer tanto las calles como los puntos de interés turísticos y construir un grafo alrededor de estos.
- El programa puede obtener caminos entre los puntos de interés, tomando el primer punto como inicial, buscando si es factible llegar al segundo punto y así hasta recorrerlos a todos.
- El programa entrega al usuario una ruta entre cada uno de los puntos, dándole direcciones exactas de como llegar y en caso de que no haya una ruta que los incluya a todos, se le comuniqué al usuario.
- Al finalizar, el programa le pregunta al usuario si quiere ingresar otro archivo txt o si desea cerrar el programa.



### 3. Modelo

#### 3.1. Interpretación del archivo de texto (mapa)

Como menciona el enunciado del proyecto, los elementos del mapa de la ciudad están ubicados en un plano XY, con coordenadas de 0 a 2000 (no negativas, es decir, solo se encuentran en el primer cuadrante).

El archivo recibido comienza con el número de calles consideradas, que debe ser menor o igual a 50. Posteriormente, se proporciona el nombre de cada una de ellas, junto con sus coordenadas de inicio y término, más el eje (X o Y) que guía la numeración de los elementos ubicados en la calle. Cada calle es una recta en el plano.

La segunda parte de este empieza con el número de puntos turísticos, para finalizar con la lista que tiene sus nombres, las calles en las que se encuentran ubicados y la posición relativa en cada calle (según el eje que guíe esta).

Desde ya podemos darnos cuenta que al momento de recorrer este mapa, es crucial considerar las intersecciones entre calles, pues es la única forma de cambiar el rumbo entre una calle y otra.

#### 3.2. ¿Cómo usaremos grafos?

Un grafo es un par ordenado  $G = (V, E)$  donde:

- $V$  es un conjunto finito y no vacío, cuyos elementos llamamos **vértices** o **nodos**.
- $E$  es un conjunto de pares no ordenados de  $V$  llamados **aristas**, que relacionan los vértices entre sí.

Para modelar el mapa como un grafo, debemos definir qué representan los nodos y las aristas.

**Nodos:** Los nodos del grafo representarán a los puntos relevantes del mapa, es decir, aquellos lugares donde ocurre algo importante para el recorrido del turista. Estos son:

- **Intersecciones entre calles:** Los únicos puntos donde el turista puede cambiar de calle. Dos calles se interceptan si sus segmentos se cruzan en el plano XY.
- **Puntos turísticos:** Son los destinos que el turista debe visitar. Se ubican en una calle a una posición determinada según el eje de numeración de esta.

Cabe destacar que un punto turístico puede coincidir con una intersección, en cuyo caso se decidió representar con un único nodo que cumple ambos roles.

**Aristas:** Consideraremos como arista a cada tramo recto de calle que el turista puede recorrer sin tomar ninguna decisión de dirección. Esto significa que dos nodos están conectados por una arista si pertenecen a la misma calle y son consecutivos en ella, es decir, no existe ningún otro nodo entre ellos en esa calle.

Existen distintas variantes de los grafos, como los dígrafos, que cuentan con **arcos** en vez de aristas, y estos pueden tener un sentido determinado. Otra variante son los grafos ponderados, donde cada arista tiene un **peso asociado**, generalmente utilizado para representar distancias o costos.

En este problema, las calles no cuentan con un solo sentido para el desplazamiento, si no que se pueden recorrer en cualquier sentido sin problemas. Por esta razón usamos un grafo normal con aristas en vez de un dígrafo.

Como existen distancias cuantificables entre cada uno de los nodos, se hace natural pensar que estas podrían representar pesos para cada arista. Sin embargo, por razones que veremos más adelante, se decidió no calcularlos al momento de construir el grafo.

#### 3.3. Construir y almacenar el grafo

La construcción del grafo se realiza en tres etapas:

1. **Agregar puntos turísticos como nodos:** Se calcula la coordenada  $(x, y)$  de cada punto turístico a partir de su calle y posición relativa. Si el eje de numeración de la calle es X, la posición relativa corresponde a la coordenada  $x$  del punto, y la coordenada  $y$  se obtiene en base al segmento de recta. Si el eje es Y, se procede de forma análoga.



2. **Calcular intersecciones:** Se recorren todos los pares de calles y se determina si sus segmentos se interceptan. Si es así, se agrega un nodo en el punto de intersección. Si en ese punto ya existe un nodo (por ejemplo, un punto turístico), se actualiza el nodo existente con la información de ambas calles, evitando duplicados.
3. **Conectar nodos por calle:** Para cada calle, se recopilan todos los nodos que pertenecen a ella, se ordenan según su posición en el eje de numeración y se conectan los consecutivos con una arista.

Dado que el enunciado establece un máximo de 50 calles, y la cantidad máxima de intersecciones entre rectas está definida por la suma de los primeros  $n$  números naturales, el límite de nodos de intersección debería ser

$$\frac{50 \times 49}{2} = 1225$$

más la cantidad de puntos turísticos, cuyo máximo no fue definido en el enunciado, pero consideraremos que serán a lo más 75 para tener un número cerrado como cantidad máxima de nodos de nuestro grafo: 1300.

Cada nodo puede pertenecer a múltiples calles. En el peor caso, si todas las calles pasan por el mismo punto, un nodo pertenecería a 50 calles, por lo que se define el máximo de calles por nodo como 50. Análogamente, cada nodo puede tener hasta 2 vecinos por calle, resultando en 100 como cantidad máxima de vecinos.

En estas condiciones, una **lista de adyacente** es ideal para representar las conexiones entre nodos. Esto se debe a que el grafo es disperso: cada nodo tiene pocos vecinos en la práctica, por lo que una matriz de  $1300 \times 1300$  desperdiciaría demasiada memoria.

### 3.4. Suposiciones sobre la idea de ruta

No olvidemos qué es lo que queremos hacer en primer lugar: encontrar una ruta que nos permita ir entre todos los puntos turísticos.

El enunciado usa intercambiadamente los términos **ruta** y **rut**as para tanto una ruta general entre todos los puntos así como para rutas entre cada par de puntos.

Nosotros interpretamos el problema con el concepto de **ruta** siendo la ruta general, de inicio a fin, abarcando todos los puntos, y **caminos** que son parte de esta ruta, entre pares de puntos, siguiendo la definición de camino con respecto a la teoría de grafos según lo visto en clases.

También interpretamos que de no haber una **ruta factible** que pase por todos los puntos (es decir, si desde alguno de los puntos turísticos no se puede hacer un camino hacia algún otro de ellos) no se creará una ruta general y se informará al usuario sobre esta situación.

### 3.5. Encontrar caminos dentro del grafo

Dejando en claro nuestras suposiciones, pasemos encontrar estos caminos entre puntos turísticos para formar la ruta.

Para obtener el camino más corto entre dos puntos en un grafo ponderado, vimos en clase el **Algoritmo de Dijkstra**. Sin embargo, como el enunciado del proyecto dice que no necesitamos una ruta óptima, descartamos la idea.

Dado que la ruta no necesita ser óptima, lo único que nos interesa es saber si existe siquiera un camino factible entre un par de puntos, es decir, si son parte de una misma componente conexa. Para esto ni siquiera es necesario considerar la distancia en el plano, por lo que descartamos incluir pesos en las aristas al momento de construir el grafo, pues el algoritmo que descubra los caminos no los usará.

Entonces, para encontrar caminos se decidió utilizar el **procedimiento de búsqueda en anchura**, conocido como BFS por su sigla en inglés (Breadth-first search). Este llega a los nodos de distancia  $k$  después de haber pasado por todos los nodos a distancia  $k - 1$ . Dado un nodo fuente  $s$ , Breadth-first search sistemáticamente explora los vértices de un grafo para “descubrir” todos los nodos alcanzables desde  $s$ , lo que nos sirve para encontrar un camino de  $s$  a cualquier otro punto del grafo.

### 3.6. Dar instrucciones al turista

Al dar instrucciones es cuando nos importan las distancias.



Utilizaremos BFS para verificar si todos los puntos turísticos son parte de la misma componente conexa, para ver si es factible una ruta.

Y si es posible, la idea es imprimir pasos por cada nodo visitado, considerando el desplazamiento dentro de una calle con metros representando las unidades de coordenada, y los puntos cardinales para indicar el sentido en el que se debe desplazar el turista.

Si desplazándose entre dos puntos turísticos se pasa por uno adicional, se considerará este visitado al momento de buscar el siguiente camino desde el punto de destino.

No se considera limitaciones respecto a la repetición de puntos turística dentro de la ruta mientras cada camino individual entre pares de puntos respete su definición.



## 4. Implementación

Para hablar de la implementación, desglosaremos los 5 archivos de código desarrollados:

### 4.1. `estructuras.h`

En este archivo, principalmente definimos los 5 tipos de struct que usaremos a lo largo del código.

- **calle:** Este se utilizara para almacenar los datos de las calles obtenidos del archivo txt. Podrá almacenar una variable tipo **char** que contendrá el nombre de la calle, cuatro variables **float** donde se guardaran el punto de inicio y los de final de esa calle (que estarán en un formato cartesiano, x,y) y por ultimo otra variable **char** que guardara el sentido que tiene esa calle (que puede ser X o Y).
- **punto:** Esta estructura se utilizara para guardar los puntos turísticos leídos en el archivo entregado por el usuario. Cada uno contendrá dos variables **char** donde se almacenara el nombre del punto turístico y otro donde se almacenara la ubicación (la calle en donde se encuentra), luego tres variables **float**, la primera para almacenar la altura en donde se encuentra, y las otras para almacenarlas como coordenadas cartesianas precisas. y por ultimo una variable **int** donde se guarda el índice de la calle.
- **Datos:** Este struct podrá almacenar dos arreglos de tipo **calle** y **punto** respectivamente, y cuatro **int** donde se almacenaran, el número de calles, el número de puntos turísticos, y la cantidad de calles que tienen sentido X o Y.
- **nodo:** Este almacenara dos variables **float** donde se almacenaran una coordenada x y una coordenada y, además de cinco variables **int**, donde una almacenara un arreglo de (como máximo) 50 calles, una donde se almacenaran el número de calles, otra variable donde se verificara si el nodo que se esta analizando es turístico, y por último un arreglo con los vecinos del nodo y el número total de estos.
- **Grafo:** Como se puede analizar por su nombre, este struct permitirá armar el grafo, este conteniendo un arreglo de **nodo** y un **int** con el número de estos.

Además, declaramos `#ifndef ESTRUCTURAS_H` y `#define ESTRUCTURAS_H`, para definir, que cumplen la función de identificar si `ESTRUCTURAS_H` esta definida o no al momento de usarlas, en caso de que no, las define, en caso de estarlo, el código omite el contenido del archivo y salta a la lineal con `#endif`.

Más después, se definen tres variables fijas, **MAX\_NODOS** con un valor de 1300, **MAX\_VECINOS** con un valor de 100 y **MAX\_CALLES\_NODO** con un valor de 50.

### 4.2. `leecturaMapas.c`

Este archivo de código se centra en recibir, abrir y leer el mapa entregado por el usuario, contando con tres funciones principales:

- **recive\_archivo:** Esta devuelve un puntero al archivo. Primero le solicita al usuario el ingresar el nombre de un archivo txt, el cual será el mapa. En caso de no poder encontrarlo, se le notifica al usuario para que ingrese correctamente el nombre del archivo. Si se logra abrir en modo lectura y no esta vacío, la función devuelve el puntero al archivo.
- **cuenta\_direcciones:** Para ser utilizada, al ser llamada se le debe entregar como parámetro un arreglo de variables tipo calle. En este se contaban cuantas tenían una dirección 'X' o 'Y', esto por medio de un ciclo for. Esta fue utilizada para las primeras versiones del código, después se descarto su uso.
- **lee\_archivo:** Se debe entregar un puntero a un archivo FILE al ser usada y devuelve una variable tipo Datos. En esta se lee linea por linea el archivo, almacenando las primeras n líneas y guardando sus datos en la variable 'mapaTuristico.calles' al leer las n calles se continua con los m puntos turísticos que se van guardando en la variable 'mapaTuristico.punto'. Además, se guardan el número de calles y puntos totales, y se cuentan cuantas calles tienen dirección X y cuantas dirección Y.



### 4.3. construyeMapa.c

En este archivo se puede primero definimos el umbral de error como **1e-5f**, para evitar problemas de precisión con floats. Luego, en el archivo contaríamos con las siguientes funciones:

- **calcular\_coordenadas\_puntos:** Esta como parámetro recibe un puntero a una variable **Datos**, la cuál debería contener lo que se extrajo del archivo de texto entregado por el usuario. En esta, se revisan los datos entregados en los parámetros, ubicando los puntos turísticos en el plano del mapa, ubicandolos en sus calles correspondientes, respetando el sentido de cada calles, en esta se esta usando además la función **fabsf()** para obtener el valor absoluto de las variables de tipo float junto al umbral para evitar errores.
- **agregar\_calle\_a\_nodo:** Esta función se le debe entregar dos parámetros, un puntero a una variable **nodo** y un **int** llamado calle. En esta función, por medio de un ciclo for, se permite que un mismo nodo reconozca que es parte de varias calles, o sea que esta en una intersección de calles. En esta además se cuida de que no haya duplicidad del nodo.
- **buscar\_nodo:** Al ser llamada, se le debe entregar un puntero de una variable **Grafo** y dos **floats**, un x y un y. Esta función se encarga de buscar un nodo en el grafo, que se encuentre en las coordenadas entregadas (x,y). Debido a estar trabajando con variables tipo **float**, se vuelve a usar el umbral para evitar errores.
- **agregar\_nodo:** Se le debe ingresar seis argumentos a la función, un puntero de una variable **Grafo**, dos variables tipo **float** que servirán como coordenadas (x,y), y tres variables **int**, las dos primeras reconocidas como c1 y c2, y la ultima que determine si el punto es turístico o no. Esta función tiene la misión de, primero reconocer si hay un nodo en las coordenadas (x,y) del grafo entregado, si ya hay uno, actualiza la información, en caso contrario lo crea y lo agrega.
- **conectar:** Al igual que otras funciones anteriores, se de debe entregar como parámetro un puntero a la variable **Grafo**, ademas de dos **int**. Esta se encarga de establecer una adyacente física entre dos puntos, revisando si dos nodos son vecinos, y en caso de serlo, registrarlos según sus indices.
- **interseccion\_segmentos** Los parámetros que se le deben entregar son: dos variables tipo **calle**, a y b, y dos punteros de tipo **float**, ix e iy. Esta función realiza el cálculo geométrico de cruces de calles, gracias al producto cruzado. Revisa si las calles son paralelas o en caso no serlo, registrar en que punto chocan.
- **construir\_grafo:** Esta función requiere como parámetros, un puntero de una variable **Datos** y de un **Grafo**. Como su nombre lo dice, en esta, se construye el grafo a utilizar. Primero declarando que tiene 0 nodos, y luego, basándose en los datos entregados, llama a las funciones anteriores para agregar más nodos, ver sus vecindades, las intersecciones de las calles, etc.

### 4.4. rutaTuristica.c

En este archivo de código, se identifican las rutas para recorrer el grafo, utilizando el algoritmo **BFS** (Breadth-First Search). Aquí contamos con tres funciones:

- **bfs:** La función que permite la búsqueda en anchura, este permite encontrar el camino entre dos nodos del grafo. Utilizando una cola para ir explorando el grafo por niveles, permitiendo encontrar una ruta entre el vértice de origen hasta el de destino. El grafo, los nodos de origen y destino, son entregados como parámetros al llamar a la función, ademas de un arreglo **int** vacío y un puntero a una variable **int** que simbolizara el largo.
- **imprimir\_ruta:** Esta función, traduce los datos matemáticos entregados para entregárselos al usuario como direcciones. En este se calcula la diferencia entre coordenadas de dos nodos consecutivos, para poder determinar un punto cardinal (Norte, Sur, Este, Oeste o diagonales). Luego toma la distancia euclidiana para obtener los metros que el turista que se encuentra en un punto, debe avanzar. Al ser ejecutada la función, va diciéndole al usuario por cuantos metros debe moverse, en que calle ir y en que dirección.



- **recorrer\_puntos:** En esta función, las otras funciones mencionadas son llamadas. Primero verifica si el camino entre dos puntos turísticos es posible, en caso de no serlo, se lo notifica al usuario. Si existe un camino, y comienza con el recorrido, marcando los nodos ya visitados, anunciando al usuario el punto de partida, la ruta a seguir, hasta el otro punto turístico, y así hasta completar todos los puntos del mapa.

#### 4.5. main.c

El archivo principal del código, en el inicio de este incluimos la librería estándar de C, además de los otros 4 archivos del código. Luego declaramos tres variables, una tipo **Grafo** llamada **g**, otra tipo **Datos** llamada **mapaTuristico** y una matriz tipo **int**, donde sus dimensiones son **x=50** e **y=MAX\_NODOS**. Ya después partiendo con la función **main**, esta inicia con un ciclo **while** infinito, donde primero se saluda al usuario, y se llama la función **recibe\_archivo** para que el usuario pueda ingresar el archivo con el mapa, para luego entregarle el puntero a la función **lee\_archivo** que lo retornado se guarda en la variable **mapaTuristico**. En caso de tener 0 calles o 0 puntos turístico se le notifica al usuario que en el mapa no hay nada que procesar. Luego cuando ya se leyó y se extrajo sus datos, se cierra el archivo y se llaman a las funciones **calcular\_coordenadas\_puntos**, **construir\_grafo** y **recorrer\_puntos**. Cuando estas finalizan, se le notifica al usuario que el mapa fue leído correctamente, y le dará dos opciones a elegir: si ingresa '1' el usuario podrá ingresar otro mapa para ser analizado, reiniciando el ciclo, o si ingresa '2' el programa se despide y se cierra. Esto se refleja en un ciclo donde se evalúa que lo ingresado por el usuario sea una opción válida, y en caso de no serlo, se le notificara que eligió una opción inválida y que elija una de las dos opciones presentadas.

#### 4.6. Compilación

Para compilar el código del programa, se debe tener GCC instalado. En la carpeta del proyecto, se debe abrir una terminal, y ingresar el siguiente comando:

```
gcc main.c -o mapa -lm
```

El nombre del ejecutable puede requerir ajustes según el sistema operativo en el que se esté compilando. Las dependencias **math.h**, **stdio.h**, **string.h** son requeridas, así como también la cabecera **estructura.h**, pero todas se encuentran enlazadas en el archivo **main**.



## 5. Conclusiones

La aplicación de los conocimientos obtenidos respecto a la teoría de grafos es esencial para la actualidad, en varias áreas como la de modelar mapas urbanos, como pudimos comprobar en el transcurso del proyecto documentado en este archivo. Al momento de escribir este código, los miembros lograron entender prácticamente los conocimientos adquiridos en las clases teóricas sobre el manejo de grafos y dígrafos, además de poder poner a prueba el algoritmo de BFS, cumpliendo con el objetivo de que el código devolviera una ruta para pasar por todos los puntos turísticos (si es posible), respetando el sentido de las calles, el nombre y cantidad de estas mismas entregadas en un archivo.

Lo realizado permitió a su vez entender el como funcionan las aplicaciones de mapas actuales y el como pueden trazar rutas desde un punto a otro, respetando los sentidos de las calles. Gracias al algoritmo escogido, se pudo obtener una ruta entre los distintos nodos (puntos turísticos) marcados en el mapa, y aunque se planteó usar el algoritmo de Dijkstra para obtener una ruta optima, por simplicidad solo utilizamos el algoritmo BFS, aunque en alguna versión futura se podría agregar. Además, al realizar el código en C, permitió un manejo más cómodo de los datos, utilizando struct para almacenar una gran cantidad de datos y permitiendo desarrollar un código más modular y legible. Finalmente, la ejecución de este proyecto no solo consolidó nuestra capacidad para traducir abstracciones matemáticas en soluciones algorítmicas funcionales, sino que también puso de manifiesto la importancia del rigor técnico en el desarrollo de software de ingeniería. El haber enfrentado y resuelto desafíos como la precisión en coordenadas reales, la gestión de datos mediante estructuras en C, o la toma de casos extremos, nos otorga una visión integral de lo que implica el modelamiento de sistemas complejos. Concluimos que el dominio de las estructuras de datos y algoritmos de trayectoria es una competencia fundamental para nuestra futura trayectoria profesional, permitiéndonos comprender y mejorar las tecnologías de navegación que definen la movilidad en el mundo contemporáneo.